

Repaso: Relaciones entre clases en Java

Haciendo un repaso de Java como lenguaje (sin tener en cuenta un framework), sabemos que, si queremos relacionar clases entre sí, podemos hacerlo de dos formas:

- . Mediante un objeto (Cuando deseamos simular el equivalente lógico a una relación 1 a 1 en una base de datos).
- . Mediante una colección de objetos (cuando queremos simular el equivalente lógico a relaciones 1 a n, n a 1 o n a n en una base de datos).

Supongamos dos ejemplos:

1. En primer lugar pensemos en una clase AUTO que tiene relación con una clase PROPIETARIO. Para este caso, está establecido que un Auto puede tener solo un Propietario. Esta relación en Java se representaría de la siguiente manera:



Como podemos ver, la relación en este caso se lleva a cabo mediante un objeto. Esto JPA lo traduciría a nivel de base de datos mediante la aplicación de una FK al propietario asociado.

Ahora supongamos otro ejemplo, donde tengamos un auto que puede tener N cantidad de propietarios:

```
public class Auto {  
  
    private Long id;  
    private String marca;  
    private String modelo;  
    private String patente;  
    private String anio;  
    private List<Propietario> listaPropietarios;  
}
```

1 Auto puede tener n
Propietarios (1 a n)

Ejemplo relación 1 a n en Java

En este caso, vemos una relación 1 a n. En estos casos, existe una particularidad en Java con respecto a las bases de datos. En bases de datos, la FK (Foreign Key) se encuentra del lado de la n, es decir, debería estar del lado de la clase Propietario; sin embargo, cuando trabajamos a nivel lógico en Java, la relación de la n se representa mediante una lista/collection desde el lado del 1, lo que luego JPA traduce y convierte al equivalente correspondiente en la base de datos.

Estos conceptos son importantes para saber cómo realizar el mapeo correspondiente de JPA y las relaciones asociadas. A continuación, se procederá a detallar cada una de las relaciones, las annotations asociadas y de qué manera podemos implementarlas.

@OneToOne

La annotation @OneToOne se utiliza en JPA + Hibernate para indicar que un atributo de una clase (que es de tipo de otra) hace referencia a una relación 1 a 1. Cabe destacar que @OneToOne se puede usar cuando una clase tiene SOLO UN ATRIBUTO del tipo de la clase a la que queremos relacionarnos.

Supongamos el primer ejemplo, el de la relación 1 a 1 que vimos anteriormente entre Auto y Propietario. Al mapear con @OneToOne, nuestro código debería verse de esta manera:

```
public class Auto {

    private Long id;
    private String marca;
    private String modelo;
    private String patente;
    private String anio;

    @OneToOne
    private Propietario unPropietario;
}
```

además del mapeo sencillo indicando la relación, @OneToOne puede ser utilizada en conjunto con la anotación @JoinColumn. Esta annotation nos permite establecer en mayor detalle con que atributo queremos relacionar la primera clase (en este caso Auto) con la segunda (Propietario) pensando como si se tratara de dos tablas en una base de datos.

Por ejemplo, entre Auto y Propietario podríamos tener algo similar a esto:

```
public class Auto {

    private Long id;
    private String marca;
    private String modelo;
    private String patente;
    private String anio;

    @OneToOne
    @JoinColumn(name = "id_propietario",
                referencedColumnName = "idPropietario")
    private Propietario unPropietario;
}
```

Nombre que queremos que tenga la FK

Nombre del atributo de la clase con la que relacionamos como PK

Donde mediante el parámetro «name» decimos el nombre que queremos que tenga nuestra FK en la tabla auto de la base de datos (id_propietario) y mediante «referencedColumnName» indicamos el nombre del atributo con el que se relacionara en la otra tabla. En este caso, estamos asociando mediante la id del propietario que sería su identificador único (PK).

Es importante tener en cuenta que la anotación @JoinColumn SOLO SE PUEDE USAR del lado de la clase que contendrá la FK (en este caso Auto).

Una vez concluido todo esto, al ejecutar nuestra aplicación, JPA + Hibernate se encargaran de crear en la base de datos las tablas correspondientes y sus debidas relaciones en base a las configuraciones que hayamos hecho.

@OneToMany + @ManyToOne

La annotation @OneToMany se utiliza en JPA + Hibernate para hacer referencia a las relaciones 1 a n. Si pensamos a nivel base de datos, se utiliza cuando una fila de una tabla necesita estar asociada a múltiples filas de otras.

Si tenemos en cuenta el segundo ejemplo entre Auto y Propietario, establecimos que un Auto puede tener muchos Propietarios. Esto lo representamos mediante una collection (List), por lo que el correspondiente mapeo lo haríamos de esta manera:

```
public class Auto {  
  
    private Long id;  
    private String marca;  
    private String modelo;  
    private String patente;  
    private String anio;  
    @OneToMany  
    private List<Propietario> listaPropietarios;  
}
```

En caso de que queramos únicamente una relación unidireccional, es decir, que solo los autos conozcan sus propietarios, es suficiente con colocar el @One ToMany.

Ahora, si estuviéramos interesados de que los propietarios también conozcan sus autos y formar así una relación bidireccional, podemos hacer uso de la annotation @ManyToOne del lado del propietario. Sin embargo, en la mayor parte de los casos, con realizar el mapeo del correspondiente @One ToMany es totalmente suficiente para lograr una relacion funcional entre clases y sus respectivas representaciones en las tablas de la base de datos.

Una vez implementada la annotation, al ejecutar nuestra aplicación, JPA + Hibernate realizarán las adaptaciones necesarias en la base de datos para que cada tabla que representa una clase de nuestra lógica tenga las relaciones y atributos necesarios.

A Dato particular: En este caso, al existir una relación uno a muchos de tipo UNIDIRECCIONAL (solo del lado de Auto), JPA + Hibernate crearán, una tabla intermedia que se encargará de interconectar a la clase Auto con la clase Propietario para que cada auto pueda conocer quiénes son sus propietarios (de forma similar a como sería una relación muchos a muchos a nivel base de datos).

Por otro lado, Hibernate establece como buena práctica que se constituya la relación bidireccional, es decir, que además de Mapear la relación @OneToMany, también mapeemos la relación @ManyToOne del lado de la clase con la que estamos asociando.

En este caso, Auto contendría la relación @OneToMany y Propietario la relación @ManyToOne; sin embargo, esta relación bidireccional exige unos parámetros extras en ambas clases:

```
public class Propietario {  
  
    private Long idPropietario;  
    private String nombre;  
    private String apellido;  
    private String dni;  
    @ManyToOne  
    @JoinColumn(name="idPropietario")  
    private Auto aut;  
}
```

En el caso de Propietario, agregaremos el @ManyToOne mediante un objeto de la clase con la que estamos creando la relación (Auto en este caso). además de eso, al igual que en una relación @OneToOne, es recomendable usar la annotation @JoinColumn, donde como nombre de la columna que utilizaremos como FK colocaremos idPropietario, para que coincida con la PK de lo que será nuestra tabla Propietario.

Una vez realizado este mapeo, es necesario agregar algo más al @OneToMany de nuestra clase Auto:

```
public class Auto {  
    private Long id;  
    private String marca;  
    private String modelo;  
    private String patente;  
    private String anio;  
    //en mappedBy colocamos el nombre del objeto  
    //con el que asociamos la relación en la otra clase  
    @OneToMany(mappedBy="aut")  
    private List<Propietario> listaPropietarios;  
}
```

Este agregado es el parámetro mappedBy. Este, busca hacer referencia de que la relación ya fue construida también por la otra clase (en este caso Propietario) a través de un objeto llamado "aut".

Una vez realizado esto, JPA + Hibernate se encargarán de llevar a cabo las adaptaciones correspondientes. En este caso, no existirá una tabla intermedia (como en la relación unidireccional) sino que se creará la correspondiente columna que alojará nuestra FK para confirmar la relación 1 a n.

@ManyToMany

Lo primero que tenemos que saber de las relaciones muchos a muchos, es que a nivel base de datos no existen «como tal»; es decir, se las implementa mediante la combinación de dos relaciones 1 a muchos y con la creación de una tabla intermedia.

En este caso, si quisiéramos una relación uno a muchos entre Auto y Propietario, Auto tendría una relación 1 a n con una tabla intermedia y Propietario también una relación 1 a n con la misma tabla. Si uniéramos imaginariamente» ambas relaciones, obtendríamos un n a n.

Sabiendo esto, la annotation @ManyToMany nos permite representar este tipo de relaciones a nivel lógico en nuestras aplicaciones desarrolladas con Java que utilicen JPA + Hibernate.

Tomando el mismo ejemplo de Auto y Propietario, el mapeo se vería de la siguiente manera:

```
public class Auto {

    private Long id;
    private String marca;
    private String modelo;
    private String patente;
    private String anio;
    @ManyToMany
    private List<Propietario> listaPropietarios;
}
```

Mapeo @ManyToMany del lado de Auto

```
public class Propietario {

    private Long idPropietario;
    private String nombre;
    private String apellido;
    private String dni;
    //agregamos el mappedBy para que asocie con la lista de Propietarios
    @ManyToMany (mappedBy="listaPropietarios")
    private List<Auto> listaAutos;
}
```

Mapeo @ManyToMany del lado de Propietario

Ahora, si bien con este mapeo que llevamos a cabo, ya es posible la creación de la tabla intermedia de forma automática y ambas clases quedarían bien representadas a nivel base de datos, existe también una annotation más que nos puede dar una mano extra si queremos ser más específicos todavía y personalizar la tabla intermedia: @JoinTable.

Como podemos ver, @JoinTable nos permite colocar el nombre que queramos que tenga la tabla intermedia, en este caso «rel_auto_propietario». y además, el nombre que tendrá cada uno de los campos que representaran las foreign keys (FK).

Automáticamente con esta personalización, JPA + Hibernate tomaran las configuraciones y luego las adaptaran en nuestra base de datos.

Unidad 6 - Relaciones JPA

```
public class Auto {  
  
    private Long id;  
    private String marca;  
    private String modelo;  
    private String patente;  
    private String anio;  
  
    @OneToMany  
    @JoinTable(  
        name = "rela_auto_propietario",  
        joinColumns = @JoinColumn (name = "FK_AUTO", nullable = false),  
        inverseJoinColumns = @JoinColumn (name="FK_PROPIETARIO", nullable = false)  
    )  
    private List<Propietario> listaPropietarios;  
}
```

Ejemplo de @JoinTable